CMPSC 465, Fall 2025
Homework 3

Mohammad Abu Shariha
ID: mma7020

Problem 1: Heap Basics

(a) Run build-heap on A to construct a min-heap

Note: All array operations use 0-based indexing.

Initial Array: $A = [23, 4, 14, 3, 9, 13, 11, 10]$. We call `min-heapify` on non-leaf nodes from index 3 down to 0.

- `min-heapify(A, 3)`: Node 3 (value 3) is smaller than its child (10). No violation, no change.
- `min-heapify(A, 2)`: Node 2 (value 14) is larger than its smallest child (11). Swap. **Array**: $[23, 4, 11, 3, 9, 13, 14, 10]$
- `min-heapify(A, 1)`: Node 1 (value 4) is larger than its smallest child (3). Swap. **Array**: $[23, 3, 11, 4, 9, 13, 14, 10]$
- `min-heapify(A, 0)`: Node 0 (value 23) is larger than its smallest child (3). Swap. The value 23 is then recursively sifted down the tree.
  1. Swap 23 and 3: $[3, 23, 11, 4, 9, 13, 14, 10]$
  2. Swap 23 and 4: $[3, 4, 11, 23, 9, 13, 14, 10]$
  3. Swap 23 and 10: $[3, 4, 11, 10, 9, 13, 14, 23]$

Final Resulting Array: `[3, 4, 11, 10, 9, 13, 14, 23]`

Problem 1: Heap Basics

(b) Insert the element 5

Starting heap: `[3, 4, 11, 10, 9, 13, 14, 23]`

- Add 5 to the end: `[3, 4, 11, 10, 9, 13, 14, 23, 5]`
- Bubble up from index 8. Parent at index 3 is 10. Since $5 < 10$, we swap.
- Array becomes `[3, 4, 11, 5, 9, 13, 14, 23, 10]`.
- The new parent at index 1 is 4. Since $5 > 4$, the heap property holds. Stop.

Resulting Array: `[3, 4, 11, 5, 9, 13, 14, 23, 10]`

Problem 1: Heap Basics

(c) Insert the element 1

Starting heap: `[3, 4, 11, 5, 9, 13, 14, 23, 10]`

- Add 1 to the end: `[3, 4, 11, 5, 9, 13, 14, 23, 10, 1]`
- Bubble up from index 9. This requires multiple swaps to reach the root:
  1. Parent at index 4 (value 9). Since $1 < 9$, swap.
     Array: `[3, 4, 11, 5, 1, 13, 14, 23, 10, 9]`
  2. New parent at index 1 (value 4). Since $1 < 4$, swap.
     Array: `[3, 1, 11, 5, 4, 13, 14, 23, 10, 9]`
  3. New parent at index 0 (value 3). Since $1 < 3$, swap.
     Array: `[1, 3, 11, 5, 4, 13, 14, 23, 10, 9]`

Resulting Array: `[1, 3, 11, 5, 4, 13, 14, 23, 10, 9]`

Problem 1: Heap Basics

(d) Delete 14

Starting heap: `[1, 3, 11, 5, 4, 13, 14, 23, 10, 9]`

- The element 14 is at index 6. Swap it with the last element (9 at index 9).
- Array becomes `[1, 3, 11, 5, 4, 13, 9, 23, 10, 14]`.
- Remove the last element (14). The effective heap is now of size 9.
- The element 9 at index 6 violates the heap property with its parent (11 at index 2). We bubble it up.
- Swap 9 and 11. Array becomes `[1, 3, 9, 5, 4, 13, 11, 23, 10]`.
- The element 9 is now at index 2. Its parent is at index `floor((2-1)/2) = 0`, which has a value of 1. Since $9 > 1$, the min-heap property holds, and we stop.

Resulting Array: `[1, 3, 9, 5, 4, 13, 11, 23, 10]`

Problem 2: Heaps and Heap Sort

(a) What are the minimum and maximum numbers of nodes in a heap of height h?

For a heap of height h:

- **Minimum nodes**: 2^h. This occurs when the tree is full to level $h-1$ and has only one node on the last level, h.
- **Maximum nodes**: $2^{h+1}-1$. This occurs when the tree is a perfect binary tree, full at every level up to h.

Problem 2: Heaps and Heap Sort

(b) Is the array with values $\{10, 14, 19, 35, 31, 42, 27, 44, 26, 33\}$ a Min heap?

No. The min-heap property requires that every parent node is less than or equal to its children. In this array, the node at index 3 (value **35**) has a child at index 8 (value **26**). Since $35 > 26$, the min-heap property is violated.

Problem 2: Heaps and Heap Sort

(c) Show that in the worst-case Heapify-UP could make $\Omega(\log n)$ swaps

The **Heapify-UP** (or sift-up) procedure is used to restore the heap property after inserting a new element. The worst case for this operation occurs when the new element must be swapped along an entire path from a leaf to the root. The length of the longest such path is the height of the tree, h.

To achieve $\Omega(\log n)$ swaps, we need an example where a newly inserted element travels all the way to the root.

Example: Consider an existing min-heap with $n = 5$ elements: $A = [10, 20, 15, 30, 40]$. We now insert a new element, **5**, which is smaller than any element in the heap.

1. **Add to end:** The new element is placed at the end of the array.
   $A = [10, 20, 15, 30, 40, \mathbf{5}]$
2. **Heapify-UP from index 5:** The element 5 at index 5 is compared with its parent at index $\lfloor (5-1)/2 \rfloor = 2$ (value 15). Since $5 < 15$, we swap.
   $A = [10, 20, \mathbf{5}, 30, 40, 15]$
3. **Continue from index 2:** The element 5 at index 2 is compared with its new parent at index $\lfloor (2-1)/2 \rfloor = 0$ (value 10). Since $5 < 10$, we swap.
   $A = [\mathbf{5}, 20, 10, 30, 40, 15]$
4. **Stop:** The element 5 is now at the root (index 0), so the process stops. The min-heap property is restored.

Analysis: For a heap with n nodes and height h, this process performs a swap at each level up the tree. The maximum number of swaps is equal to the height of the tree. Since the height of a heap with n elements is $h = \lfloor \log_2 n \rfloor$, the number of swaps in the worst case is $\Theta(\log n)$. This directly satisfies the $\Omega(\log n)$ lower bound.

Problem 3: Median of Streaming Data

We can find the median of a data stream in $O(\log n)$ time per element by using two heaps:

- A **Max-Heap** (`lower_half`) to store the smaller half of the numbers.
- A **Min-Heap** (`upper_half`) to store the larger half of the numbers.

We maintain the invariants that the heaps are always balanced (sizes differ by at most 1) and that every element in `lower_half` is less than or equal to every element in `upper_half`.

Pseudocode

```
function process_new_number(x, lower_half, upper_half):
    // 1. Insert new element
    if lower_half.is_empty() or x <= lower_half.peek_max():
        lower_half.insert(x)
    else:
        upper_half.insert(x)

    // 2. Rebalance the heaps
    if lower_half.size() > upper_half.size() + 1:
        element_to_move = lower_half.extract_max()
        upper_half.insert(element_to_move)
    else if upper_half.size() > lower_half.size():
        element_to_move = upper_half.extract_min()
        lower_half.insert(element_to_move)

function get_median(lower_half, upper_half):
    if lower_half.size() == upper_half.size(): // Even number of elements
        return (lower_half.peek_max() + upper_half.peek_min()) / 2
    else: // Odd number of elements
        return lower_half.peek_max()
```

Time Complexity Analysis

For each new number, the algorithm performs at most two heap insertions and one heap extraction. Each of these operations on a heap of size $\approx n/2$ takes $O(\log(n/2)) = O(\log n)$ time. Calculating the median takes $O(1)$ time. Thus, the complexity is $^{**}O(\log n)$ per number received**.

Problem 4: Proofs

Statement: Show that an n-element binary heap has height $\lfloor \log_2 n \rfloor$.

Proof: A binary heap is a complete binary tree. Let n be the number of nodes and h be the height.

1. A complete binary tree of height h has its first h levels (0 to $h-1$) completely full. The number of nodes in these levels is $2^h - 1$. To achieve height h, there must be at least one node on level h. Thus, the minimum number of nodes is $n_{min} = (2^h - 1) + 1 = 2^h$. This gives us the lower bound:

$$n \geq 2^h$$

2. The maximum number of nodes for a tree of height h is a perfect binary tree, which has $2^{h+1} - 1$ nodes. This gives us the upper bound:

$$n \leq 2^{h+1} - 1$$

3. Combining these two inequalities, we have:

$$2^h \leq n \leq 2^{h+1} - 1$$

Because $n \leq 2^{h+1} - 1$ implies $n < 2^{h+1}$, we can write the stricter inequality:

$$2^h \leq n < 2^{h+1}$$

4. Taking the logarithm base 2 of all parts of the inequality:

$$\log_2(2^h) \leq \log_2(n) < \log_2(2^{h+1})$$
$$h \leq \log_2(n) < h + 1$$

5. By the definition of the floor function, if h is an integer such that $h \leq x < h + 1$, then $h = \lfloor x \rfloor$. Therefore, the height h of an n-element heap is:

$$h = \lfloor \log_2 n \rfloor$$